



PROG7311 POE PART 3

Technical Reflection Report

Abstract

Global Logistics Management System

ST10445500

BCAD Year 3

Table of Contents

Introduction.....	2
DevOps + Automated Testing.....	3
Containerisation + Docker.....	5
Conclusion	7
References	8

Introduction

The final part of the GLMS POE focused on taking the working prototype and modernising it into a more enterprise-style application. The project was no longer only a single MVC system. It was split into a separate ASP.NET Core Web API backend, an ASP.NET Core MVC frontend, an SQL Server database, automated tests, and a Docker Compose setup. This changed the project from a simple local application into a system that is easier to test, run, and maintain.

This reflection focuses on two important parts of the final POE requirement. The first part is DevOps and automated testing, especially why automated testing is important in a CI/CD pipeline and how it helps prevent bugs from reaching production. The second part is containerisation, especially how Docker helps keep development, testing, and production environments consistent and solves the common “it works on my machine” problem.

In GLMS, these two areas are connected. The system has important business rules, such as preventing service requests from being created against expired or on-hold contracts, validating PDF uploads, converting foreign currency values into ZAR, and making sure the MVC frontend correctly communicates with the API. If these areas break, the system may still open in a browser, but the business workflow would no longer be correct. This is why automated testing and containerisation are not just extra features. They help prove that the system works in a repeatable way.

DevOps + Automated Testing

DevOps is about making development, testing, and deployment work together more smoothly. Instead of leaving testing until the end, CI/CD allows the project to be checked continuously as the code changes. GitHub describes GitHub Actions as a CI/CD platform that can automate build, test, and deployment pipelines (GitHub Docs, 2026). This is important because it means code can be checked automatically instead of relying only on a developer remembering to manually test every screen before submission.

For GLMS, the GitHub Actions workflow was used as a CI check. The workflow runs restore, build, test, and Docker image build checks when changes are pushed or opened as pull requests. This means the project is checked in a clean build environment, not only on my own computer. If the solution does not restore, does not build, or the tests fail, then the workflow fails. This creates a quality gate before the code is treated as ready.

The GLMS test project uses xUnit and includes both unit tests and integration tests. The unit tests are focused on important business logic. These include contract service validation, service request rules, blocking service requests for expired and on-hold contracts, PDF file validation, currency conversion calculations, and repository filtering behaviour. These tests are useful because they check the rules directly and quickly. For example, if a future code change accidentally allows a service request to be created against an expired contract, the relevant test should fail before the issue reaches the submitted or deployed version.

The integration tests test the API at a broader level. Microsoft explains that integration tests check whether app components work correctly together, including areas such as the request-response pipeline, database, file system, and network-related behaviour (Microsoft, 2026a). In GLMS, the integration tests use WebApplicationFactory to make HTTP-style calls to the API. This is closer to how the MVC frontend actually uses the backend, because the frontend does not directly connect to SQL Server anymore. It sends HTTP requests to the API and receives JSON responses.

This matters because a method can work on its own while the full endpoint still fails. For example, a service method could be correct, but the controller route could be wrong, the API could return the wrong status code, the dependency injection setup could be broken, or the JSON response could be shaped incorrectly. Integration tests help catch these problems because they test the API more like a real client would use it. In GLMS, this includes checking status codes, JSON responses, and end-to-end flows such as creating data and then reading it back.

The integration tests also use an InMemory database, fake authentication, and a fake currency service. This keeps the tests repeatable. The tests do not need a real SQL Server container to be running, and they do not need the external currency API to be online for every test run. That is important in a CI/CD pipeline because tests must be reliable. If tests depend too heavily on live external services, then a temporary internet or API issue could make the build fail even when the project code is correct.

Automated testing does not mean testing every single line of code. For a student project, that would be too much and could make the test suite harder to understand. The better approach for GLMS was to test the most important logic and the workflows that have the biggest impact on the system. This includes the rules that protect the business process: contracts, service

requests, signed agreements, and currency conversion. This is also closer to how an enterprise project would prioritise tests, because the most important risks should be checked first.

How Automated Testing Prevents Bugs from Reaching Production

Automated testing prevents bugs from reaching production by moving bug detection earlier in the development process. If testing is only done manually at the end, then bugs are found late, when they are more difficult and stressful to fix. In a CI/CD process, tests can run every time code is pushed. This gives quick feedback to the developer and reduces the chance that broken code is merged or deployed.

In GLMS, automated tests help prevent several types of bugs. The first type is business rule bugs. The main example is the service request workflow. A logistics manager should not be able to create a service request if the parent contract is expired or on hold. If this validation breaks, TechMove could accidentally process work against an invalid agreement. That would be a serious workflow and compliance problem. A test for this rule acts as a guard around that business requirement.

The second type is file handling bugs. GLMS allows signed agreement PDFs to be uploaded and downloaded. This feature must reject invalid files such as executable files and should only accept valid PDF uploads. If this validation breaks, the system could store unsafe or incorrect files. The file validation tests therefore protect both the business requirement and the basic safety of the upload process.

The third type is financial calculation bugs. GLMS converts foreign currency values into ZAR using an external exchange-rate API. Even if the API connection works, the local calculation still needs to be correct. A calculation mistake could lead to incorrect service request costs being saved or displayed. The currency conversion tests help check that a known amount and a known rate produce the expected ZAR value.

The fourth type is API communication bugs. In Part 3, the MVC frontend was decoupled from the database and changed to call the Web API instead. This means the API became the main service layer of the system. If an endpoint returns the wrong status code, broken JSON, or a failed response, the frontend may not be able to complete the workflow. The integration tests help catch these issues by calling endpoints and checking the responses.

The GitHub Actions workflow improves this further because it runs the checks outside my local development setup. A common issue in software projects is that the developer thinks everything works because it runs locally, but the code fails on another machine. CI helps reduce that risk. If the build and tests pass in GitHub Actions, it gives better evidence that the project is not only working because of hidden local settings on my computer.

This is how automated testing prevents bugs from reaching production or final submission. It does not magically remove every bug, but it creates a repeatable safety net. Each important rule becomes something that can be checked again and again. If a future change breaks that rule, the test fails immediately. That failure is useful because it stops the mistake before the system is treated as ready.

Containerisation + Docker

Containerisation was the second major part of the Part 3 modernisation. Docker allows an application and its dependencies to run inside containers. Docker's own documentation explains that containers contain what is needed to run the application, so the application does not rely as heavily on what is installed directly on the host machine (Docker Inc., 2026a). This is the main reason Docker helps with consistency.

The "it works on my machine" problem happens when an application runs on the developer's computer but fails on another computer. This can happen because of different tool versions, missing SDKs, different SQL Server versions, wrong ports, environment variables, or missing folders for uploaded files. In GLMS, this could be a serious issue because the system has three main runtime parts: SQL Server, the API backend, and the MVC frontend. Without Docker, each of these parts would need to be configured separately.

GLMS uses Docker Compose to run the full system together. Docker Compose is used for defining and running multi-container applications (Docker Inc., 2026b). In the GLMS setup, Compose starts the SQL Server database container, the ASP.NET Core Web API container, and the ASP.NET Core MVC frontend container. These containers are defined as services in the `docker-compose.yml` file.

The GLMS Docker setup uses three main containers. The `glms-db` container runs SQL Server 2022. The `glms-api` container runs the ASP.NET Core Web API. The `glms-web` container runs the ASP.NET Core MVC frontend. These containers communicate through Docker's internal network. The API connects to the database using the service name `glms-db`, and the frontend connects to the API using the service name `glms-api`. This is cleaner than using hardcoded local machine addresses because the containers can find each other by their service names inside the Docker network.

Docker also helps with persistent data. GLMS uses Docker volumes so that important data does not disappear every time a container restarts. The `sql-server-data` volume stores the SQL Server data, while the `contract-uploads` volume stores uploaded signed agreement PDFs. This is important because the database and uploaded agreement files are part of the system's state. Without volumes, restarting or recreating containers could remove important data.

The Docker setup makes the project easier to run for development and marking. Instead of manually starting SQL Server, then the API, then the MVC frontend, the whole stack can be started using Docker Compose. This reduces setup mistakes. It also makes the project more realistic because enterprise systems often have multiple services that need to run together.

For Dev, Test, and Prod environments, Docker improves consistency because the same application image can be built and then reused in different environments. Development can use Docker Desktop locally, testing can build and run the same images in a pipeline, and production can use the same container concept on a server or cloud platform. The environment values may change, such as connection strings or secrets, but the application image and runtime setup stay consistent. Docker explains that containers are portable and can run on a developer's laptop, physical or virtual machines, cloud providers, or mixed environments (Docker Inc., 2026a).

This does not mean the local Docker setup is automatically production-ready. In a real production system, I would still need stronger secret management, HTTPS certificates,

database backup strategy, monitoring, health checks, and probably a more advanced hosting environment. However, the Docker Compose setup is still a strong step for this POE because it proves that the app can run as a connected ecosystem and not only as separate projects inside Visual Studio.

How Docker Solves the “It Works on My Machine” Problem in GLMS

Docker solves the problem by making the environment part of the project setup. In GLMS, the database, backend, and frontend are described in the Compose file. This means the marker or another developer does not need to guess which database version to use, which services to start first, or how the API and frontend should find each other.

For example, if another developer pulls the repository, they can use the Docker Compose setup to build and run the same services. The database container uses the configured SQL Server image. The API container uses the API Dockerfile. The web container uses the frontend Dockerfile. The internal Docker network connects the services. This is much more repeatable than relying on each person’s local computer setup.

It also supports the architectural changes made in Part 3. Since GLMS was split into GLMS.Api and GLMS.Web, the two projects needed a reliable way to communicate. Docker Compose helps here because it creates a defined network between the containers. The web app can call the API by service name, and the API can call the database by service name. This confirms that the final architecture is not just separated in code but also separated at runtime.

Docker and automated testing also support each other. The tests prove the code logic and API behaviour, while Docker proves the application can run in a repeatable environment. Together, they reduce two different risks. Automated testing reduces the risk of broken logic reaching production, and Docker reduces the risk of the app failing because the environment is different.

Conclusion

The final GLMS project uses automated testing and Docker containerisation to support a more reliable enterprise-style application. Automated testing is critical in a CI/CD pipeline because it gives fast and repeatable feedback whenever the code changes. In GLMS, the tests protect the most important business rules, including contract validation, service request restrictions, PDF upload validation, currency conversion, repository filtering, and API endpoint behaviour. The GitHub Actions workflow strengthens this by running checks in a clean environment instead of only relying on my local machine.

Docker solves the environment consistency problem by packaging and running the database, API, and web frontend as containers. Docker Compose defines the full system in one setup, allowing the SQL Server database, API backend, and MVC frontend to run together with internal networking and persistent volumes. This directly solves the “it works on my machine” problem because the project becomes easier to run consistently across development, testing, and future production-style environments.

Overall, automated testing and Docker were not only added to meet the POE checklist. They improved the quality of the final system. The tests help prove that the application logic is still working, and the Docker setup helps prove that the full stack can run in a repeatable way. This makes the final GLMS submission more stable, more maintainable, and closer to the type of structure expected from a modern enterprise web application.

References

Docker Inc. (2026a). Docker overview. [online] Docker Docs. Available at: <https://docs.docker.com/get-started/docker-overview/> [Accessed 6 Jun. 2026].

Docker Inc. (2026b). Docker Compose. [online] Docker Docs. Available at: <https://docs.docker.com/compose/> [Accessed 6 Jun. 2026].

GitHub Docs. (2026). Understanding GitHub Actions. [online] GitHub Docs. Available at: <https://docs.github.com/en/actions/get-started/understand-github-actions> [Accessed 6 Jun. 2026].

Microsoft. (2026a). Integration tests in ASP.NET Core. [online] Microsoft Learn. Available at: <https://learn.microsoft.com/en-us/aspnet/core/test/integration-tests?view=aspnetcore-10.0&pivots=xunit> [Accessed 6 Jun. 2026].

Microsoft. (2026b). Testing in .NET. [online] Microsoft Learn. Available at: <https://learn.microsoft.com/en-us/dotnet/core/testing/> [Accessed 6 Jun. 2026].